ETH zürich

D-INFK Lehre



MLOps

Lamberto Ragnolini, Carlos Cotrini
AI and IT in industry

AGENDA

Three questions, one pipeline

1

Why do we need MLOps

What goes wrong when a model meets the real world, and what it looks like when it does not.

2

What is MLOps

Where it comes from, how to define it, and how mature a team can be.

3

How we do MLOps: the pipeline

Data, training, evaluation, deployment and monitoring, with a focus on serving models that are small enough to run anywhere.

Data ingestion | Data preparation | Model training | Model evaluation | Model deployment | Monitoring

PART 1 OF 3

01

Why do we need MLOps

Models are the small part. What surrounds them decides whether they work.

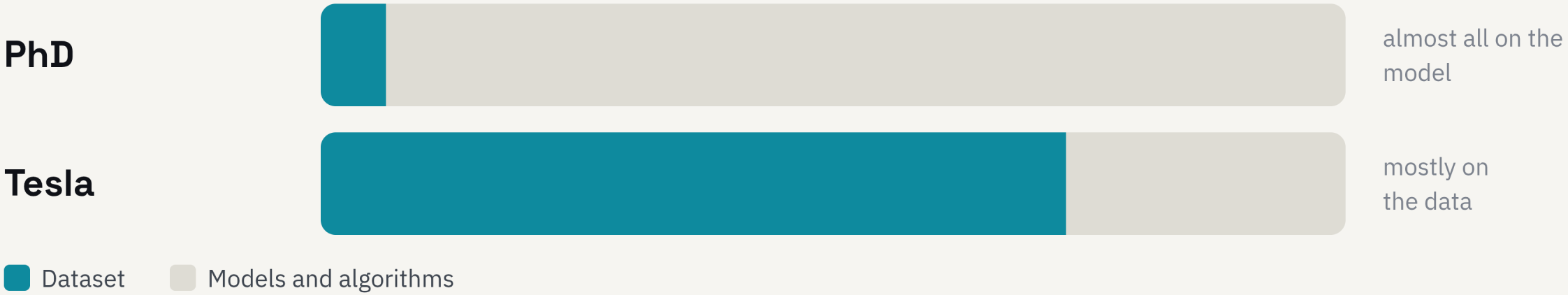
Why we need it

What it is

How we do it: the pipeline

Where the effort goes: research versus industry

Andrej Karpathy on the sleep he lost as a PhD student and at Tesla



> **In production, the work moves from the model to everything around it.**

A. Karpathy, Building the Software 2.0 Stack, Spark+AI Summit 2018. Proportions as drawn in the talk, illustrative.

90%

of ML failures come not from **poor models** but from **poor production practices**.

McKinsey, "MLOps: so AI can scale", 2024

Zillow bets a company on one number

Zestimate

A model that puts a price on every home in the United States and shows it on every listing.

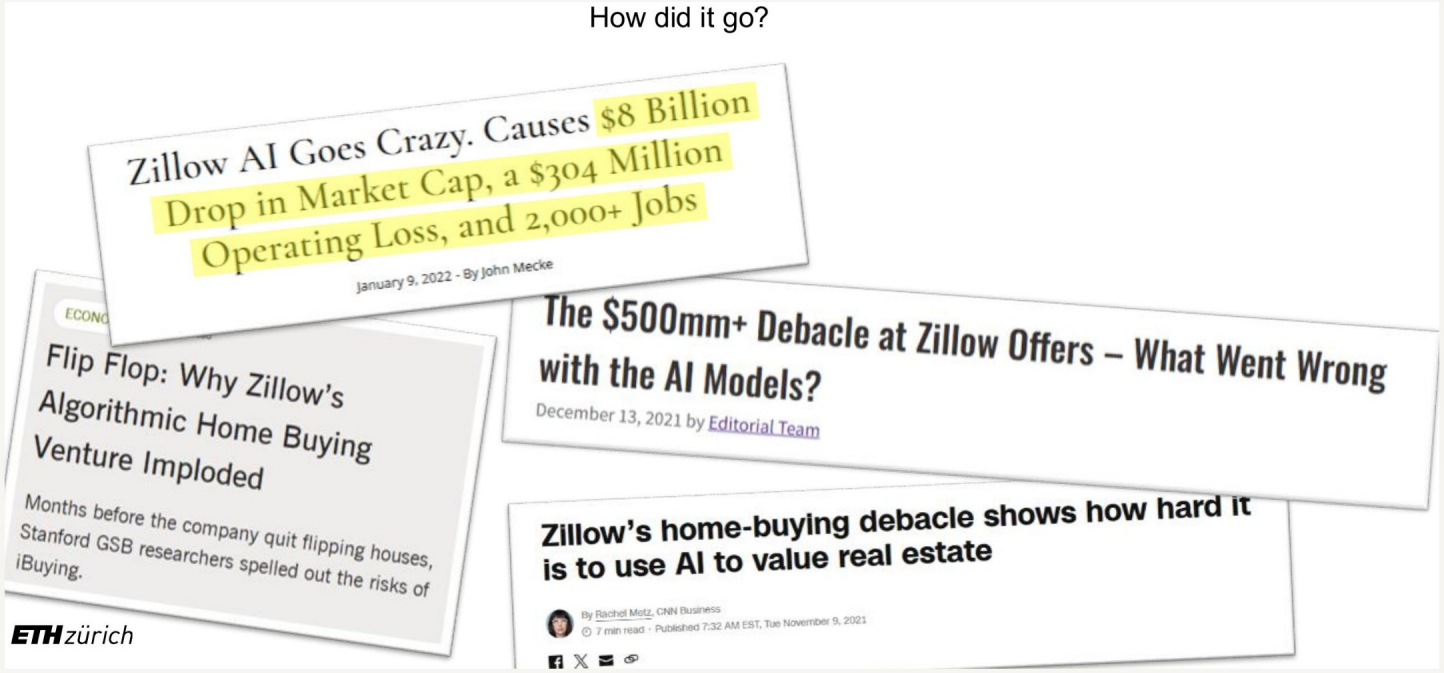
Zillow Offers, 2018

Buy houses at the price the model says, renovate them, sell them on at a profit.

How did it go?

Zillow Offers ran from 2018 to November 2021

It did not go well



$304M
operating loss in one quarter

2,000+
jobs cut

$8B
drop in market value

Headlines as shown in the FS26 lecture; figures as reported in the press, November 2021 to January 2022

What it looks like when it works

Uber's Michelangelo: one ML platform shared by every team since 2016

~400
active ML projects

5,000+
models in production

10M
predictions per second at peak

20K+
training jobs every month

> **Same decade, same kinds of models. The difference is everything around them: feature store, registry, training, serving and monitoring built once.**

Uber Engineering, "From Predictive to Generative: How Michelangelo Accelerates Uber's AI Journey", May 2024

The model is the small box



5%

of a mature ML system is learning code, at most

95%

is the glue that keeps it running, at least

Sculley et al., "Hidden Technical Debt in Machine Learning Systems", NeurIPS 2015, Figure 1

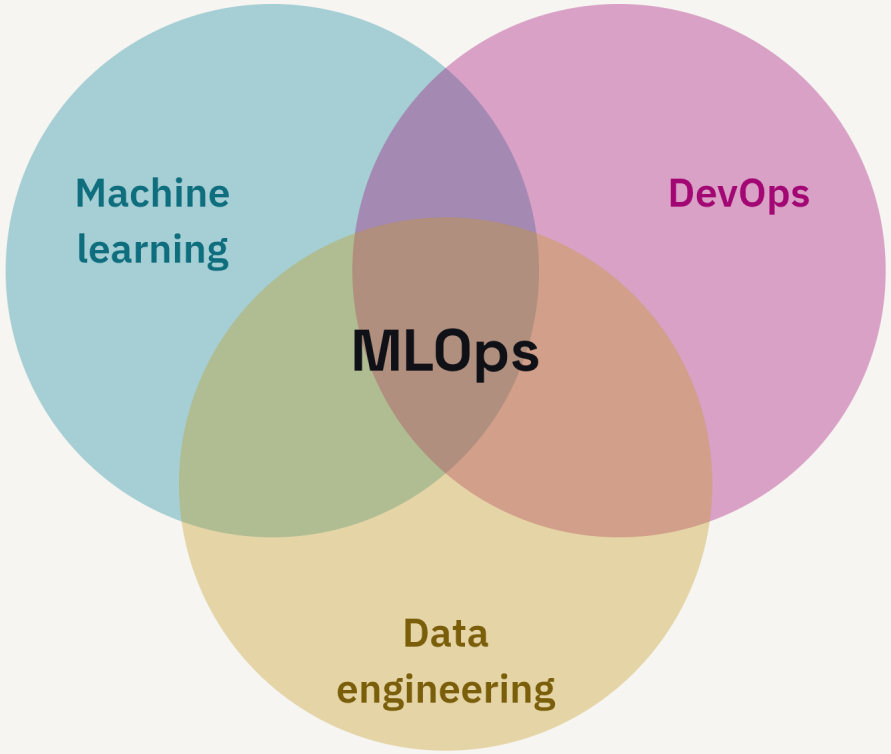
PART 2 OF 3

02

What is MLOps

DevOps, plus two artifacts that software never had: data and trained models.

Three disciplines, one practice



MLOps is an engineering practice that leverages three contributing disciplines: **machine learning**, **software engineering, especially DevOps**, and **data engineering**.

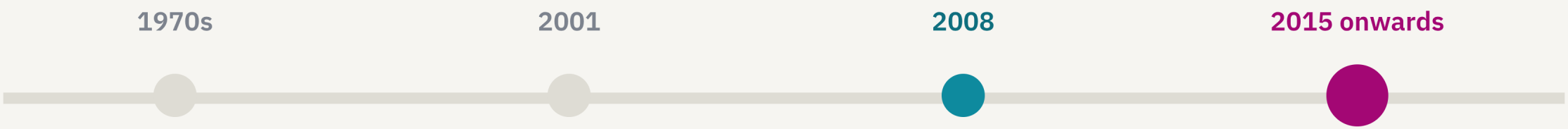
It aims at productionising ML systems by bridging the gap between development and operations.

Data engineer: pipelines, production readiness

Data scientist: analysis, modelling, insight

Kreuzberger, Kühl, Hirschl, "MLOps: Overview, Definition, and Architecture", IEEE Access 2023

Where it comes from



Waterfall

plan, build, ship once

Agile

short iterations, fast dev, slow ops

DevOps

one pipeline: CI/CD, tests, monitoring

MLOps

the same pipeline, plus data and models

> **At which step did the artifact being shipped change? With MLOps it is no longer only code.**

Liang et al. 2024 (arXiv 2405.09819) §2.1; Kreuzberger et al. 2023; Sculley et al. 2015

Nine principles behind every MLOps setup

P1

CI/CD automation

Every change to code, data or model is built, tested and deployed automatically.

P2

Workflow orchestration

The pipeline steps run in order, as a DAG, on a schedule or a trigger.

P3

Reproducibility

Same data, same code, same config: the same model, every time.

P4

Versioning of data, code, model

Each has versions you can compare, audit and roll back to.

P5

Collaboration

Scientists, engineers and ops work on shared, visible artifacts, not e-mail attachments.

P6

Continuous training and evaluation

Retrain on new data, and promote only what passes the evaluation gates.

P7

Metadata tracking

Every run logs its parameters, metrics, artifacts and environment.

P8

Continuous monitoring

Watch data, model and system in production for drift and failures.

P9

Feedback loops

What monitoring finds flows back into data collection and the next training round.

Kreuzberger, Kühl, Hirschl, IEEE Access 2023, principles P1 to P9

What ML engineers actually optimise for: the three Vs

V

Velocity

ML is experimental. Go from a new idea to a trained model in a day, not a month.

V

Validation

Errors get expensive once users see them. Test changes and prune bad ideas as early as possible.

V

Versioning

You cannot anticipate every bug. Keep many versions of models and data to debug and roll back.

> **Interviews with 18 ML engineers: it is fine for most experiments never to reach production, as long as ideas can be tried and validated fast.**

Shankar, Garcia, Hellerstein, Parameswaran, "Operationalizing Machine Learning: An Interview Study", 2022

How mature is your team? Three levels

Level 0

Manual

Notebooks and scripts, hand-offs by e-mail

A few releases a year

Models silently break in production

Level 1

Automated training

The training pipeline is code and runs on a trigger

Data and model validation gates

Frequent retraining, same pipeline

Level 2

CI/CD for ML

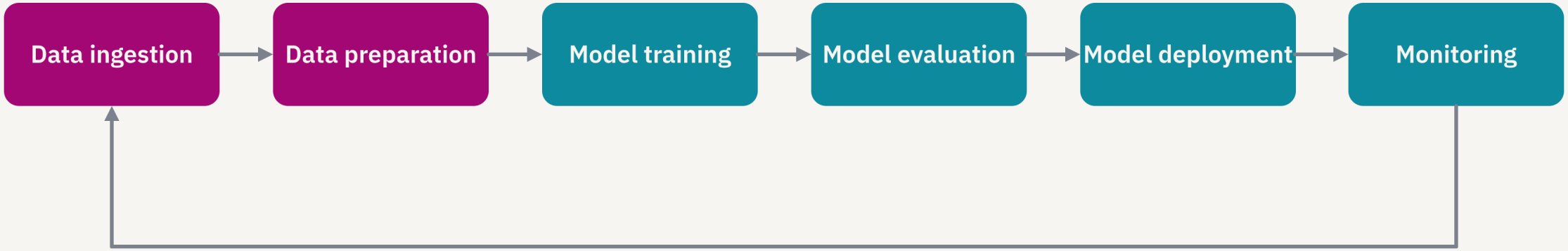
The pipeline itself is built, tested and deployed

New features, models and pipelines ship daily

Needs a platform team

Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning"

The pipeline we follow for the rest of the lecture



new data, new requirements, new model: the loop never closes

Data **Model** Two stages are about data, four about the model. In practice the stages run as a loop, not a line.

Our running example: Trendora

CASE STUDY

Trendora

Fashion that follows the flow

Retail, clothing and accessories

Physical stores and an online shop. Amsterdam, Berlin, Barcelona.

The promise

Each location reflects what is trending locally, thanks to data-driven merchandising and machine learning.

The ask

Improve operations using AI. We will return to Trendora at every stage of the pipeline.

PART 3 OF 3

03

How we do MLOps: the pipeline

Six stages, one loop. We walk through each of them, with Trendora along for the ride.

Why we need it

What it is

How we do it: the pipeline

STEPS 1 AND 2 OF THE PIPELINE

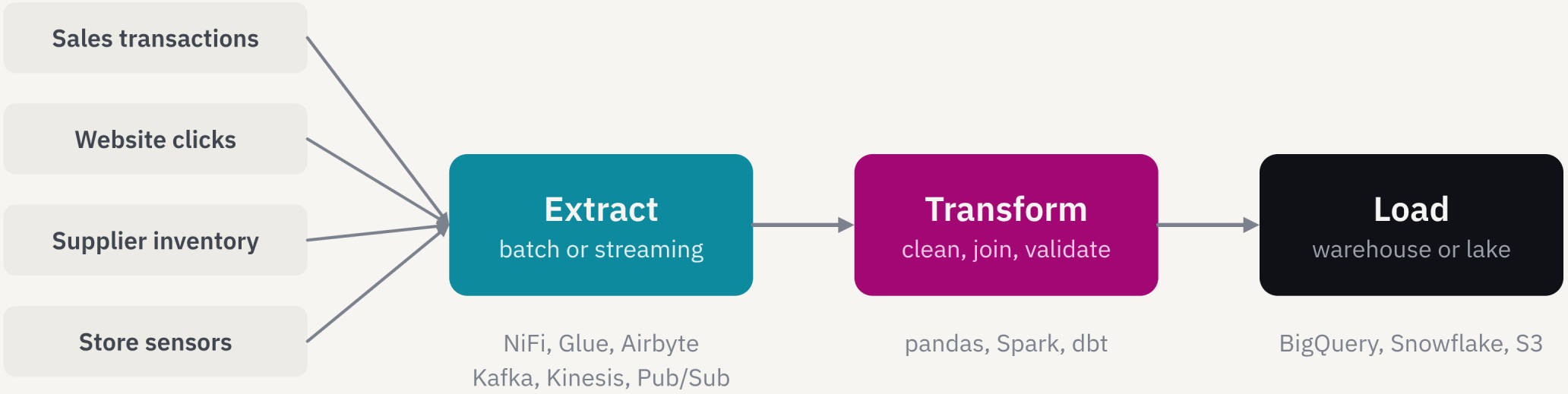
01

Data

Bringing raw data in, and making it fit to learn from.

Data ingestion | **Data preparation** | Model training | Model evaluation | Model deployment | Monitoring

ETL: how raw data gets into the ML system



> **Ingestion must be automated, cleaned and on time, like keeping shelves stocked. Nothing downstream is better than what arrives here.**

01 · DATA INGESTION

Batch or streaming

Batch



Groups of records loaded at a time: last night's sales, this week's inventory.

Apache NiFi · AWS Glue · Airbyte

Streaming



Records saved continuously as they happen: clicks, sensor readings, card payments.

Apache Kafka · Spark Streaming · Kinesis, Pub/Sub

Where the data lands

Data warehouse

clean shelves, labelled boxes

Only structured, validated data

Fast, reliable queries for BI and reports

BigQuery, Snowflake, Redshift

Data lake

a bit of everything

Structured and unstructured, with or without schema

Cheap object storage; scientists pick what they need

S3, Azure Data Lake

Data lakehouse

half warehouse, half lake

Any type of data, and fast queries on it

One platform for analysts and scientists

Databricks, Delta Lake, Iceberg

Garbage in, garbage out

RAW

Height	BP
165	120/50
6ft	NaN
180	NaN
175	125/71
10	10000

PREPARED

Height	BP_1	BP_2
165	120	50
183	120	70
180	120	70
175	125	71

Cleaning

Duplicates, missing values, outliers

Format fixing

Dates, units, currencies: €12.99 is not 12.99

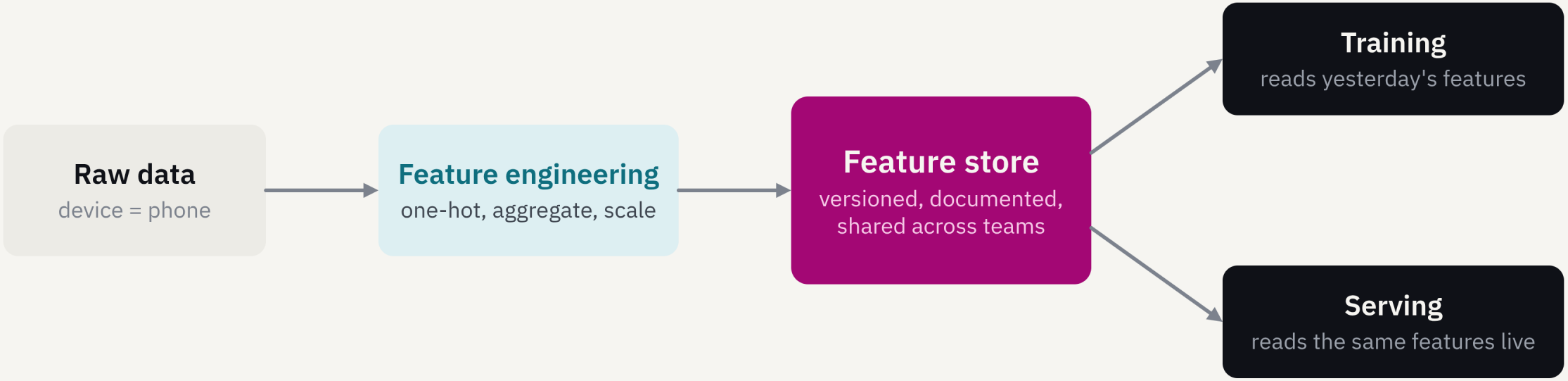
Standardisation

One name per thing: T-shirt is tee is tshirt

Validation rules

Business constraints: prices are positive, 50 states means 50

Feature engineering, and why features need a store



> **Training-serving skew, a feature computed one way offline and another way online, is the most important and least implemented test in Google's ML Test Score.**

Breck et al., "The ML Test Score", IEEE Big Data 2017; Uber Palette feature store, 2024

Train, validation, test



Train

Fits the parameters of the model.

Validation

Compares algorithms and hyperparameters trained on the training set. Independent, unbiased.

Test

Approximates performance in the wild. Looked at once, at the very end.

> **Never, under any circumstance, look at test performance before selecting a model. Keep the distributions similar and watch for leakage across the split.**

STEP 3 OF THE PIPELINE

03

Model training

Can you reproduce the model you shipped? If not, you cannot fix it either.

What every training run must record

Data

Dataset version or hash, and the exact split

DVC, LakeFS, Delta

Code

The commit of the training code and features

Git

Config

Hyperparameters, feature list, random seed

MLflow, W&B

Environment

Container image, library versions, hardware

Docker, Conda lock

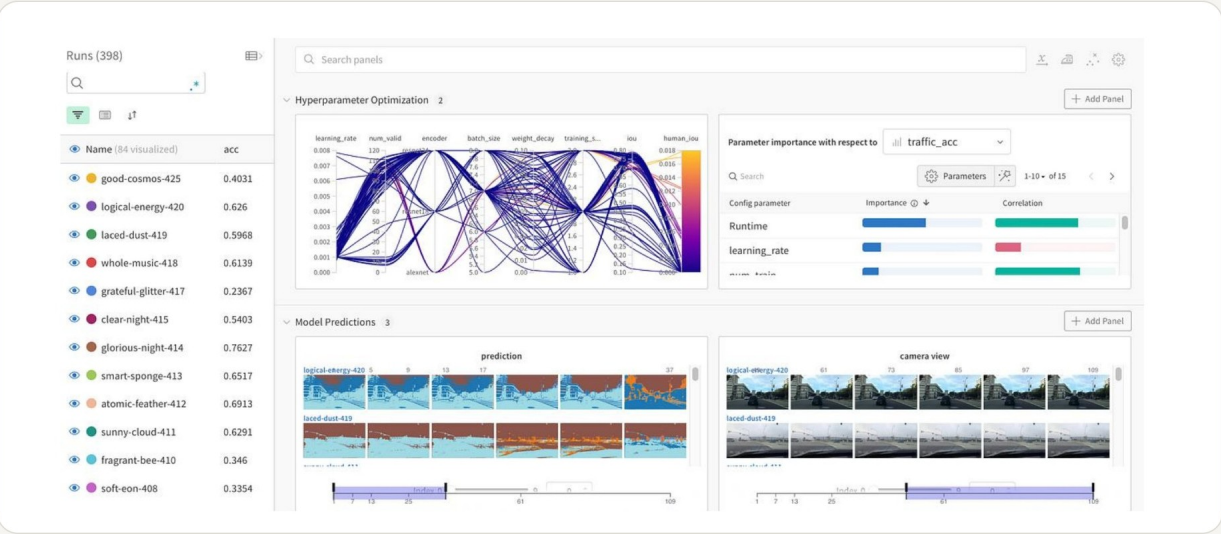
Outputs

Metrics, overall and per slice, artifacts, plots

Model registry

The pipeline writes these down, not a person. Anything manual is skipped on the day it matters.

Experiment tracking: from paper to a tool



Weights and Biases (W&B), an experiment-tracking platform. **MLflow**, used in the exercise, does the same.

Pen and paper

Level 0. Every run is a line in a notebook. Lost by Friday.

A spreadsheet

Level 0. Better, still manual, still nobody else can find it.

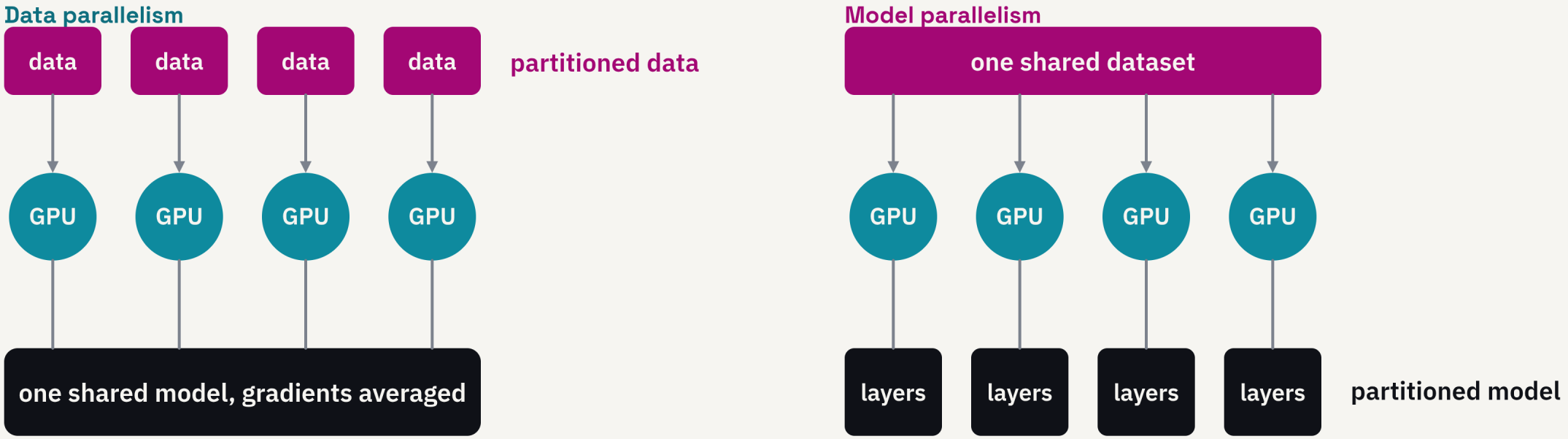
A tracking tool

Level 1 and 2. Every run logs itself: params, metrics, artifacts, environment.

> **Tracking turns 398 runs into a searchable, comparable, reproducible history, and lets a team work on the same problem.**

Screenshot: Weights and Biases, as shown in the FS26 lecture

Distributed training: split the data, or split the model



> **Facebook, 2018: data parallelism gave 4x the throughput using 5x the machines. Scaling is never free.**

Hazelwood et al., "Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective", HPCA 2018

But I do not have hundreds of machines

Cloud computing: on-demand network access to a shared pool of computing resources, provisioned and released with minimal effort.

On-demand self-service | Broad network access | Resource pooling | Rapid elasticity | Measured service

AWS
Amazon

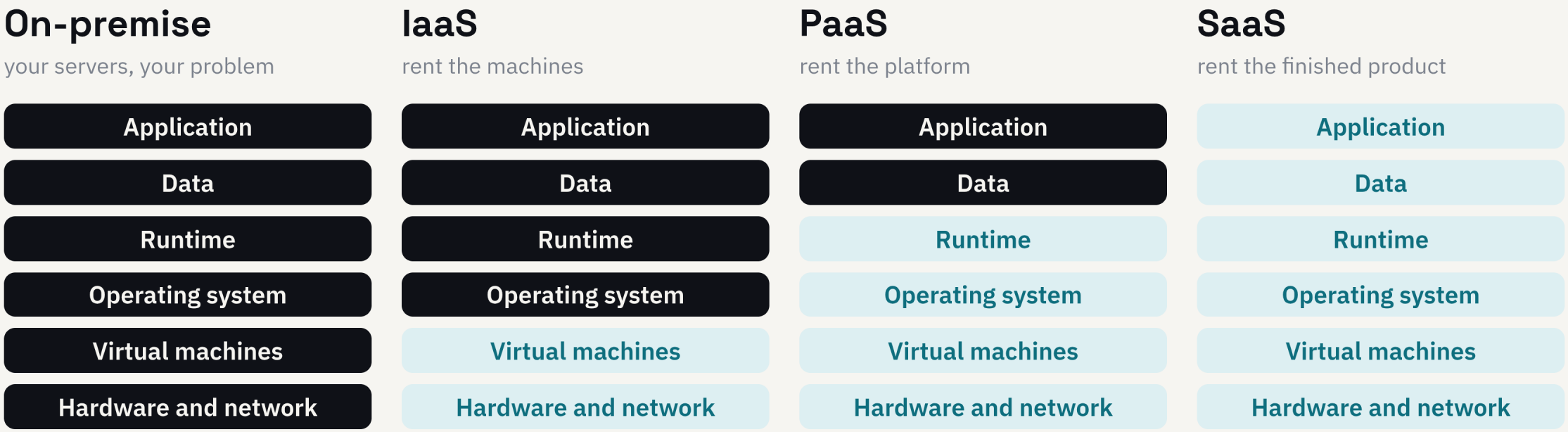
Azure
Microsoft

GCP
Google

Pay for what you use, when you use it. Keeping everything on your own servers is called on-premise deployment.

Mell and Grance, "The NIST Definition of Cloud Computing", NIST SP 800-145, 2011

Cloud computing: how much of the stack do you still manage?



ETH zürich

Service models after NIST SP 800-145: the consumer does not manage the infrastructure but controls what sits above it

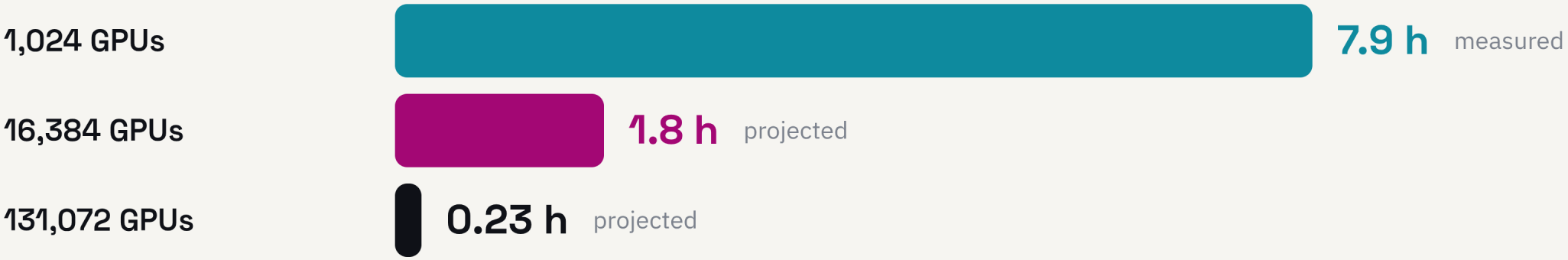
At scale, failure is the normal case

4M
jobs over 11 months on Meta's research clusters

150M+
A100 GPU-hours

19%
of GPU runtime hit by hardware failures

MEAN TIME TO FAILURE OF ONE JOB



> **Checkpoint often, expect restarts, and treat the training pipeline as infrastructure, not a script.**

Kokolis, Kuchnik et al., "Revisiting Reliability in Large-Scale Machine Learning Research Clusters", Meta, 2024

STEP 4 OF THE PIPELINE

04

Model evaluation

A good offline score is a health check, not a result.

Data ingestion | Data preparation | Model training | **Model evaluation** | Model deployment | Monitoring

Offline model performance is just a health check

−0.1

correlation between offline model gain and business gain across 23 comparisons of 46 models at Booking.com

Offline metrics

Accuracy, AUC, RMSE on a held-out set. Fast, cheap, and blind to what users do.

Online metrics

Conversion, revenue, satisfaction in a randomised controlled trial. Slow, expensive, and the only thing that counts.

Latency is a metric too

About 30% more latency cost more than 0.5% in conversion. A better model that is slower can lose.

Bernardi, Mavridis, Estevez, "150 Successful Machine Learning Models: 6 Lessons Learned at Booking.com", KDD 2019

Evaluation does not end offline

1

Offline

Held-out test set, per-slice metrics, bias checks. Cheap. Catches the obvious.

2

Shadow

The new model sees real traffic, its answers are logged, nobody sees them. Catches skew and crashes.

3

Canary and A/B

A small share of users, then more, against the champion. Measures business impact, with error bars.

4

Full rollout

The challenger becomes the champion. Monitoring takes over from here.

> **Each stage is a gate. A model that fails one goes back, it does not go forward. The release mechanics come in the monitoring section.**

Shankar et al. 2022: staged deployments typically have one to four stages, the first often at 1% of traffic

STEP 5 OF THE PIPELINE

05

Model deployment

Deploying is easy if you ignore all the hard parts.

Data ingestion | Data preparation | Model training | Model evaluation | **Model deployment** | Monitoring

Online or batch prediction

Online

Predictions generated and returned as soon as a request arrives. Served from an endpoint.

- fraud check during a payment
- recommendations
- chatbots
- search ranking

Batch

Predictions generated periodically or on a trigger, for inputs you know in advance. Written to a table.

- churn score for every customer
- demand forecast
- credit scoring for all users

100 to 400 ms

extra search latency cost Google 0.2 to 0.6% of daily searches per user

+30%

latency cost Booking.com more than 0.5% in conversion

Definitions: C. Huyen, Designing Machine Learning Systems, ch. 7. Latency figures: Google 2009, Booking.com KDD 2019

Where does the model live?

Batch job

a step in the data pipeline

Runs on a schedule in Spark, Airflow or a cloud job

Output lands in a table or the feature store

Cheap: hardware only while it runs

Server endpoint

an API behind a load balancer

Model packaged in a container, scaled by a serving framework

Mostly CPUs at scale: Facebook ran tens of trillions of inferences a day on CPUs in 2017

Latency is the budget

On the device

phone, camera, smart mirror

No network round trip, data stays local

Limited memory, compute and battery

Needs a small model: the rest of this section

Hazelwood et al., HPCA 2018: inference primarily on CPUs; training on GPUs and CPUs

What a serving framework does for you

Batching

Collects requests that arrive together and runs them as one batch, so the GPU is never idle. Triton calls it dynamic batching.

Concurrency and routing

Several models and versions on one machine, A/B and canary traffic splits, model pipelines.

Observability

Throughput, latency, GPU utilisation and error metrics out of the box, ready for the monitoring stage.

2 to 4x

more throughput for LLMs at the same latency with vLLM's paged KV cache

Before paging, only 20 to 38% of the memory reserved for the attention cache held real tokens. Serving large models is a memory-management problem as much as a compute one.

NVIDIA Triton · TorchServe · vLLM · Hugging Face TGI

Kwon et al., "Efficient Memory Management for LLM Serving with PagedAttention", SOSP 2023; NVIDIA Triton Inference Server docs

Build your own MLOps stack, or buy one?

Build

For

Full control and flexibility. Open-source building blocks: MLflow, Kubeflow, Airflow, Metaflow.

Against

High cost and maintenance burden, slower time to market, needs specialised people and mature DevOps.

Buy

For

Fast deployment, proven reliability, integrates with the cloud you already use: SageMaker, Vertex AI, Azure ML, Databricks.

Against

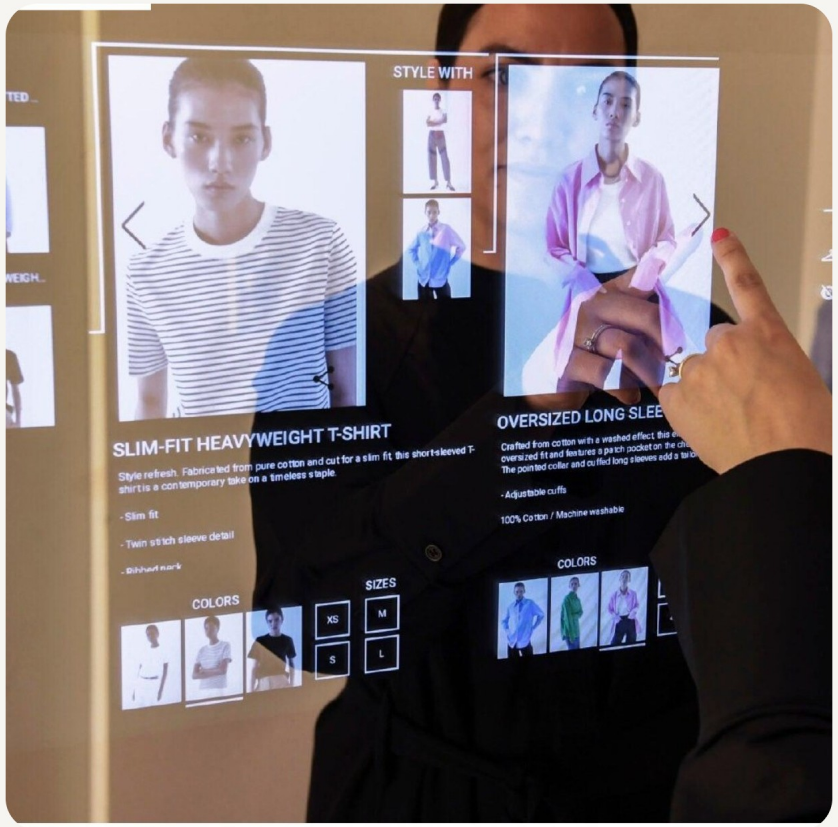
Less customisation, vendor lock-in, ongoing licence or usage costs, data leaves your walls.

> **Build buys control, buy buys time. Know which one you need.**

Tagliabue et al., "Reasonable Scale Machine Learning with Open-Source Metaflow", 2023; Uber Engineering 2018, 2024

05 · MODEL DEPLOYMENT

ML at the edge



Trendora's smart mirrors should answer in milliseconds and keep faces inside the store.

10x
less video to the cloud: the drone runs the detector on board

96%
less data moved: a wind farm summarises sensor data on site

36%
lower cost in that wind-farm deployment

PII
faces and names never leave the device

> **But the device is small. The model has to shrink: compression.**

ETH zürich

Murshed et al., "Machine Learning at the Network Edge: A Survey", ACM Computing Surveys 2022 · Photo: Jon Kopaloff, Getty Images for Cos

Four ways to make a model smaller

A 100-million-parameter model at 32 bits per weight is 400 MB before it does anything. A phone, a camera or a cost budget asks for less. Each knob trades a little accuracy for a lot of size or speed.

Low-rank factorisation

Replace a big weight matrix by the product of two thin ones. Fewer numbers, same shape.

Knowledge distillation

Train a small student to imitate a large teacher. The student learns from the teacher's doubts.

Pruning

Find the weights, neurons or layers that contribute least and remove them, then retrain.

Quantisation

Store and compute with fewer bits: 32-bit floats become 8-bit or 4-bit integers.

Taxonomy after C. Huyen, Designing Machine Learning Systems, ch. 7

Low-rank factorisation: two thin matrices instead of one fat one



$m \cdot n$

parameters before

becomes

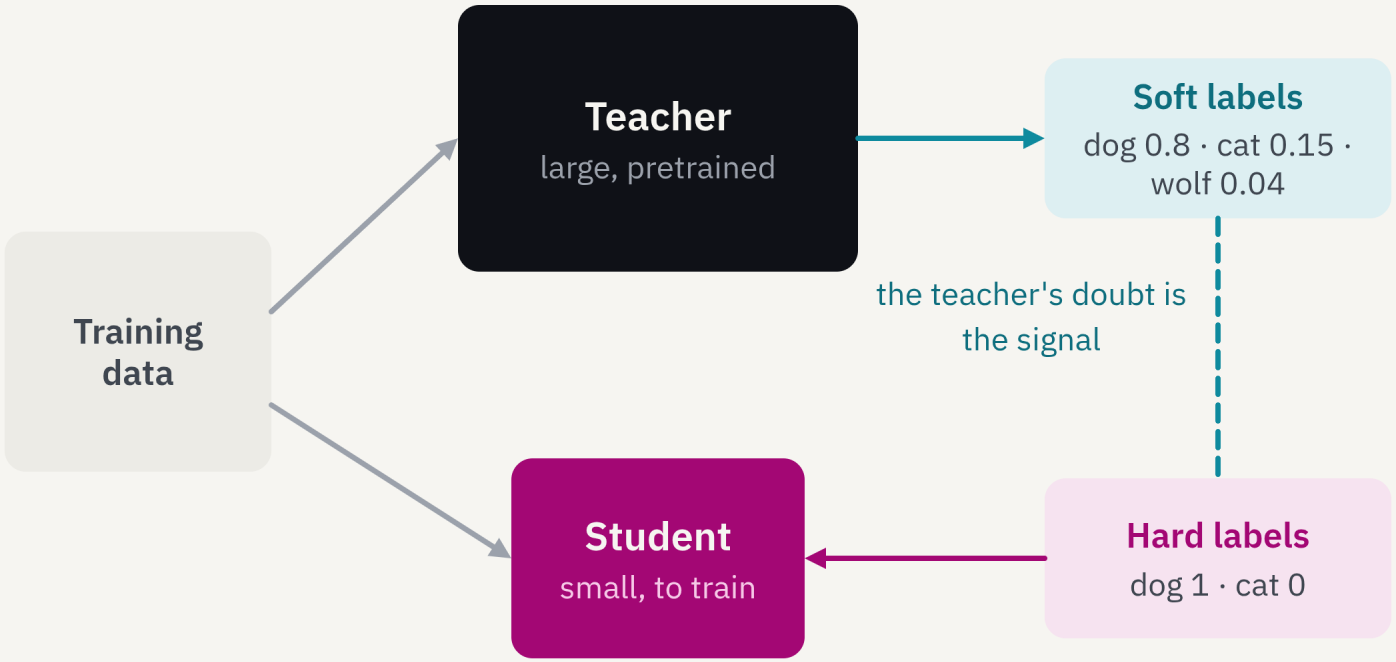
$k \cdot (m + n)$

parameters after, a big saving when k is much smaller than m and n

It is an approximation. SqueezeNet reached AlexNet-level accuracy on ImageNet with 50 times fewer parameters by rethinking the layers this way.

Iandola et al., SqueezeNet, 2016; C. Huyen, Designing Machine Learning Systems, ch. 7

Knowledge distillation: a small model learns from a big one



40%
smaller than BERT

60%
faster at inference

97%
of its language understanding kept

Hinton, Vinyals, Dean, 2015 · Sanh et al., DistilBERT, 2019

Distillation in practice, from 2015 to today

10 → 1

Google, 2015

An ensemble of 10 speech models and one distilled model reach the same word error rate, 10.7%, against 10.9% for a model trained the usual way.

20x

LinkedIn, 2025

A recommendation LLM made more than 20 times smaller than its 100B-parameter teacher with no appreciable loss. Distilling cost 0.06% AUC where plain fine-tuning cost 0.62%.

14B

DeepSeek, 2025

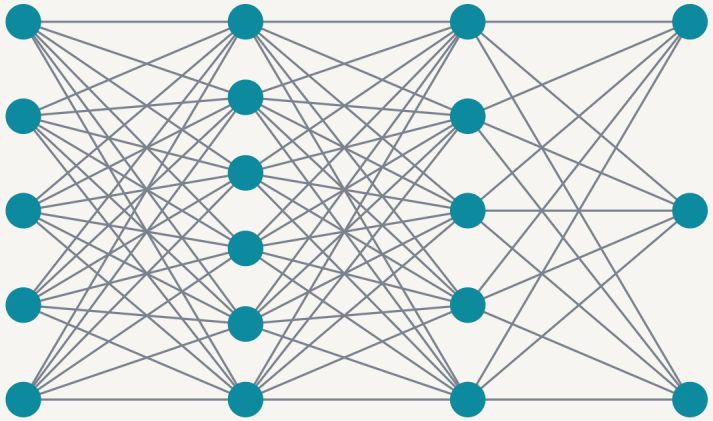
A 14B student trained on 800k teacher outputs scores 69.7 on AIME 2024; OpenAI's o1-mini scores 63.6. Distilling beat training the small model with RL directly.

> **Lesson from LinkedIn: find the smallest student that meets the quality bar, and a bigger teacher is not always a better one.**

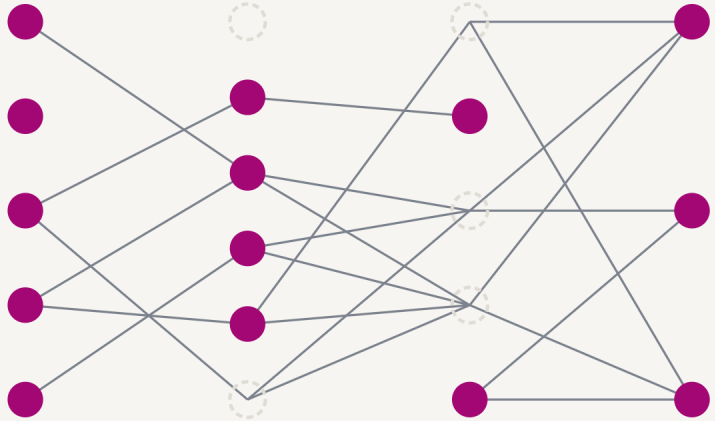
Hinton et al. 2015, Table 1 · Behdin et al., arXiv 2502.14305 · DeepSeek-AI, arXiv 2501.12948, Table 5

Pruning: most weights are not pulling their weight

BEFORE



AFTER: TRAIN, PRUNE, RETRAIN



9x

fewer parameters in AlexNet, 61M to 6.7M, same accuracy

13x

fewer in VGG-16, 138M to 10.3M, same accuracy

What to remove: single weights below a threshold, whole neurons, or whole layers. LinkedIn found the last layers of an LLM the safest to drop and the first the most damaging.

Han, Pool, Tran, Dally, NeurIPS 2015 · Behdin et al., LinkedIn, 2025

Quantisation: the same numbers in fewer bits

FLOAT32: 640 BITS

3.1134	**2.44**	**−1.569**	**8.2222**
1.234	5.045	−3.003	1.002
3.43	2.1145	2.44	2.13
1.0345	4.0313	3.2121	3.2564
2.0471	0.9982	−1.112	6.4053

round(x / S + Z)

INT8: 160 BITS

3	**2**	**−2**	**8**
1	5	−3	1
3	2	2	2
1	4	3	3
2	1	−1	6

Each weight becomes a small integer with one scale and one offset per tensor. Four times less memory, integer arithmetic, and a rounding error the network barely notices.

> **16 to 8 bits halved inference memory for a 175B model with no loss. At 3 to 4 bits GPTQ still reports negligible degradation and about 3.25x speed on an A100.**

Dettmers et al., LLM.int8(), NeurIPS 2022 · Frantar et al., GPTQ, ICLR 2023

What it buys: BERT at a billion requests a day, on CPUs



30x

lower latency and higher throughput than vanilla BERT

3,000+

inferences per second on one 36-core CPU server

1B+

inferences a day in production, median latency under 20 ms

> **Distillation, then smarter inputs, then quantisation. Each step alone was not enough; together they turned a research model into a product on cheap hardware, 6x cheaper per inference than a GPU.**

Roblox Engineering, "How We Scaled BERT To Serve 1+ Billion Daily Requests on CPUs", 2020

LinkedIn job search: the whole bill

7B → 0.6B
teacher distilled into a student

→ 375M
after pruning half the MLP neurons and 8 layers, under 1% quality loss

3.15M
items to score per second

Shorter inputs

A learned summariser cut job descriptions by 93%, prompts from 900 to 220 tokens.

Serving engineering

Batch tokenisation, no decode step, one removed CPU-GPU sync took p99 latency from 6.2 s to 0.45 s.

Caching

A 15-minute score cache answers over half of online requests.

10x
throughput versus the first version, 2,000 items per second per GPU

+1.49%
weekly active users in the online A/B test, with 8.45% better ranking

Behdin et al., "Scaling Up Efficient Small Language Models Serving and Deployment for Semantic Job Search", LinkedIn, 2025

Adapting a big model without retraining it: LoRA



10,000x

fewer trainable parameters than full fine-tuning of GPT-3 175B

3x

less GPU memory during training

0

added inference latency: A·B is merged back into W

> **The same low-rank trick, applied to the update instead of the weights.**

Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models", ICLR 2022

STEP 6 OF THE PIPELINE

06

Monitoring and continual learning

The model is live. Now the world starts changing under it.

Data ingestion | Data preparation | Model training | Model evaluation | Model deployment | **Monitoring**

“

We don’t have a good idea of how the model is going to behave in production until production.

An ML engineer at a large company, interviewed by Shankar et al.

Shankar, Garcia, Hellerstein, Parameswaran, "We Have No Idea How Models will Behave in Production until Production", CSCW 2024

My system works. What can go wrong?

Software failures

loud, familiar, usually fast to spot

Dependency
a library or service changes under you

Deployment
the wrong artifact ships

Hardware
a node, a disk, a GPU

Downtime
the endpoint is simply not there

ML failures

silent, specific, found late

Production data differs from training data
the distribution moved

Edge cases
inputs the training set never had

Degenerate feedback loops
the model's own outputs shape its next inputs

Training-serving skew
features computed differently online and offline

Shankar et al. 2024: ML bugs just silently cause errors

What drifts, and where to look for it

Covariate shift

the inputs change

New kinds of customers arrive. A store opens in a new city.

Label shift

the outcomes change

Returns double during the sale season.

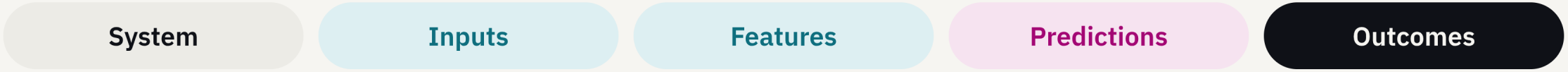
Concept drift

the relation changes

Same customer, same features, but they stop buying what the model recommends.

fast, cheap, usually an engineering bug

slow, expensive, closest to the business



> **Labels arrive weeks late or never, so watch inputs and predictions first, outcomes when you can.**

C. Huyen, Designing Machine Learning Systems, ch. 8 · Booking.com, KDD 2019

06 · CONTINUAL LEARNING

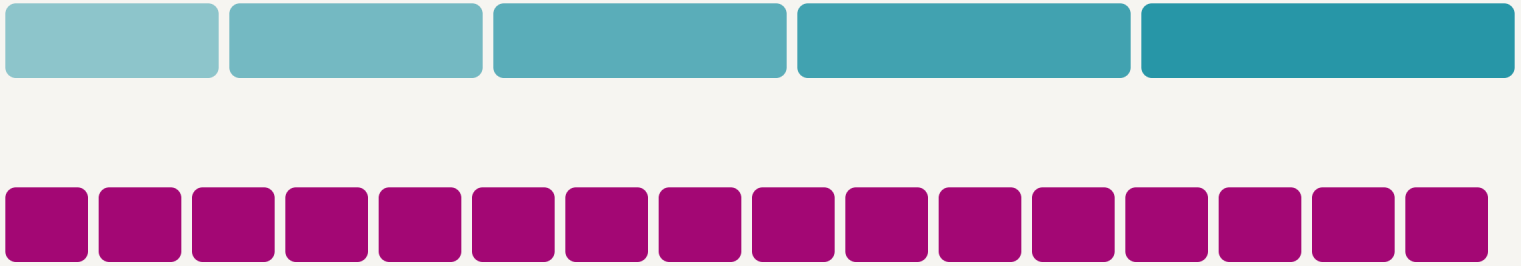
How often should I retrain, and from where?

Stateless

retrain from scratch on a growing window of data

Stateful

continue from yesterday's weights on the new data only



~1%

lower loss at Facebook from retraining daily instead of weekly

He et al., ADKDD 2014

45x

cheaper training at Grubhub by going stateful, with a 20% metric gain

Egg, RecSys 2021

minutes

between parameter syncs in ByteDance's online-trained recommender

Monolith, 2022

> **Ask how fresh the data must be, what a retrain costs and what it gains. Then measure.**

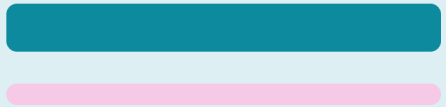
Four ways to let a new model in

Champion / challenger



The new model has to beat the one in production before it replaces it.

Shadow



Both models see every request. Only the live answer is shown; the shadow's is logged.

Canary



A few percent of traffic first, then more, rolled back if it misbehaves.

Blue / green

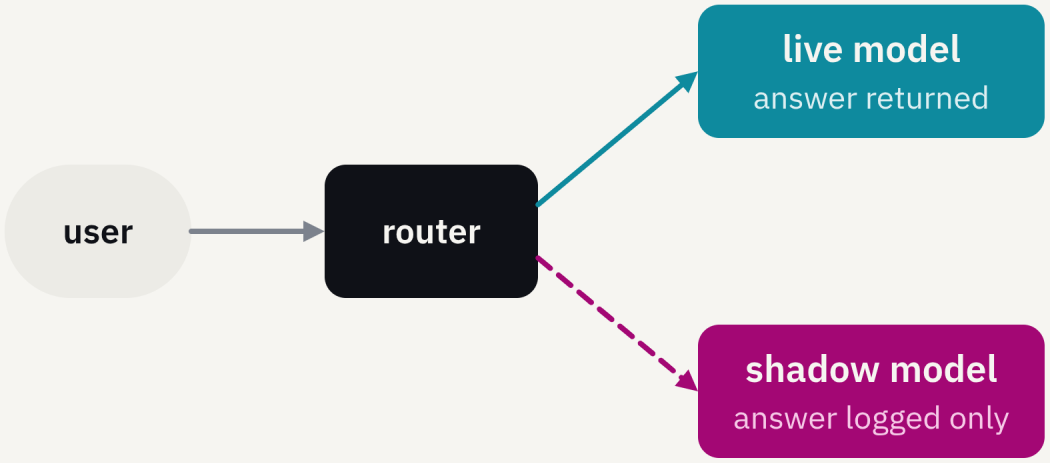


Two identical environments. Switch all traffic at once, switch back just as fast.

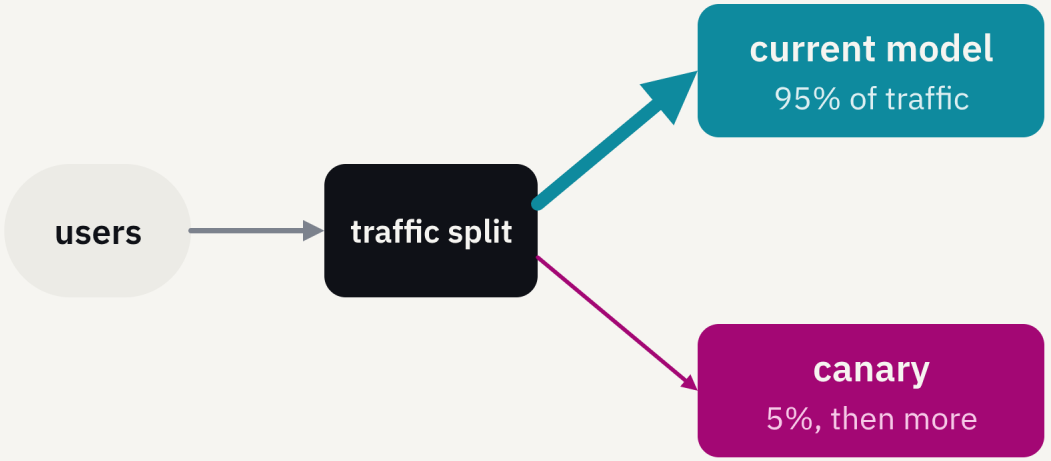
Shadow tests, canary measures, blue/green rolls back. They combine.

Shadow and canary, side by side

Shadow: test in production, nobody notices



Canary: a few users notice first



> **Shadow costs double the compute and finds skew and crashes. Canary costs some users a worse model and finds what only users can tell you.**

Why continual learning is hard

Fresh labelled data

You need labels as fast as the data. A proper ingestion and labelling pipeline, or natural labels from clicks and purchases.

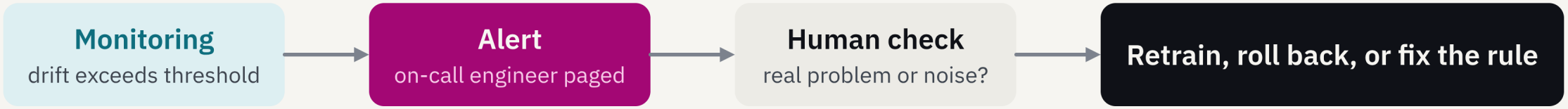
Evaluation

Updating often makes catastrophic failures more likely and opens the door to poisoned data. Every update must pass the gates, automatically.

Some models cannot

Trees and matrix factorisation do not take one small batch of new data gracefully. Neural models do, with care for forgetting.

Alerts, humans, and ML that operates ML



Alert fatigue is real

One interviewed engineer's alert fired about 1,000 times and was ignored 45% of the time. Another: you typically ignore most alerts. Alert only on signals somebody will act on.

Netflix: rules first, then a model

56%

of memory-configuration job failures fixed with no human

50%

lower compute cost for those jobs

300 hand-written rules still left half of failures unclassified; a small network took the second pass.

Shankar et al. 2022 · Netflix Technology Blog, ML-powered auto remediation in the Netflix data platform, March 2024

TRENDORA

How Trendora does all of this

DATA INGESTION

From stores and websites via Kafka into a data lake.

DATA PREPARATION

Metrics aggregated per item, features extracted and saved to the feature store as Parquet.

MODEL TRAINING

Data scientists try XGBoost; every version is tracked. The old recommender is retrained on new data.

MODEL EVALUATION

A classic computer-vision technique looks promising. Not fancy, but fast, and it passes the gates.

MODEL DEPLOYMENT

The model is quantised and deployed on some of the store's smart cameras.

MONITORING

Canary release: the new feature runs first in Berlin, then Amsterdam, then everywhere, watched all the way.

> **Use case: a recommendation system that spots what customers in each city like, built on the full pipeline.**

ETH zürich

TOOLING

MLflow: one open-source toolset for most of this

```
import mlflow
with mlflow.start_run():
mlflow.log_params(params)
mlflow.log_metric("f1", f1)
mlflow.sklearn.log_model(model, "model")
```

Five lines turn a script into a tracked, reproducible run. This is what the exercise does.

Tracking

Parameters, metrics, artifacts and environment for every run

Projects

Packaged, reproducible code with its dependencies

Models

One format to deploy to batch, endpoint or edge

Registry

Versions, stages and lineage: who promoted what, when

mlflow.org. Alternatives: Weights and Biases for tracking, Kubeflow or Metaflow for pipelines, Airflow for orchestration

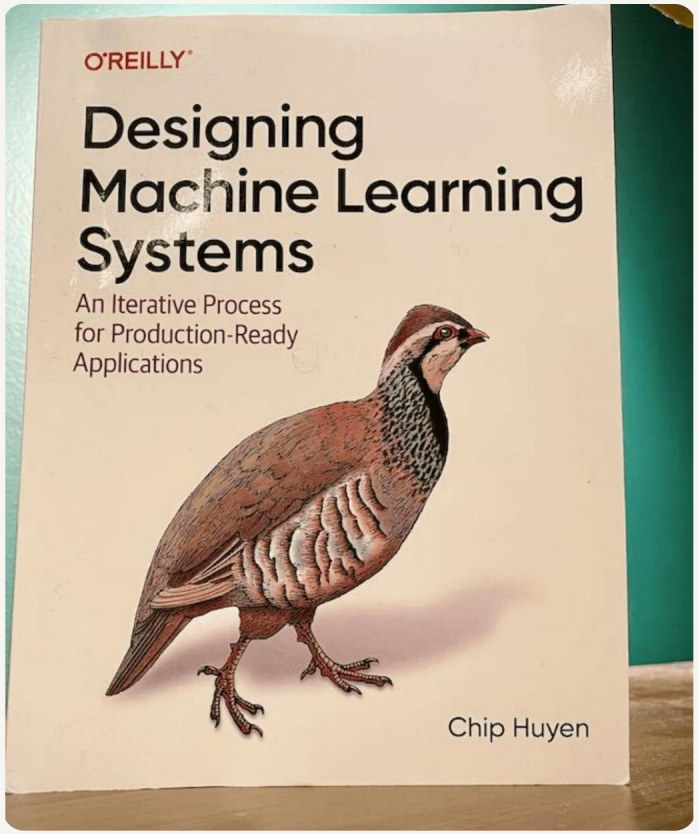
SUMMARY

What MLOps really means

1. The model is the small box. Most failures, and most of the work, are in the data and the infrastructure around it.
2. Reproducibility is the foundation: data, code, config, environment and outputs are recorded by the pipeline, not by a person.
3. Offline metrics are a health check. Shadow, canary and A/B tests are the exam.
4. A model that cannot run where it is needed is not deployed. Distill, prune and quantise until it fits.
5. Production is where the world changes. Monitor every layer, retrain on purpose, and keep humans in the loop.

FURTHER READING

One book covers the whole lecture



Designing Machine Learning Systems

Chip Huyen, O'Reilly, 2022

The lecture follows its structure: data, training, evaluation, deployment, monitoring, and the loop that connects them. The compression and drift chapters are the ones to read first.

huyenchip.com/books